

31. sim GPS 軌跡ノード

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/gps_sim_node.py
種別	Python
区分	runtime node
役割	planner/speed_cmd を積分して route waypoints 上の現在位置を求め、/sim/gps を publish する。
起動文脈	sim モードの地図上位置源。

このファイルを読む理由

planner/speed_cmd を積分して route waypoints 上の現在位置を求め、/sim/gps を publish する。

- 純粋に速度指令から位置を進める。

呼び出し関係

どこから呼ばれるか

- launch/solarcar_sim.launch.py

次に読むと理解が進むファイル

- mpc_solarcar/solar_state_node.py

同一リポジトリ内の主要依存

- mpc_solarcar/path_utils.py
- mpc_solarcar/route_utils.py
- mpc_solarcar/solar_profile.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from .path_utils import resolve_path
from .solar_profile import require_csv_data_rows

class GPSSimNode(Node):
    def __init__(self):
        super().__init__('gps_sim_node')
        self.declare_parameter('route_csv', 'inputs/route_waypoints.csv')
        self.declare_parameter('dt', 1.0) # Hz
        self.declare_parameter('init_speed_kmh', 40.0)
        route_csv = self.get_parameter('route_csv').value
        route_csv = resolve_path(route_csv, 'inputs')
        route_csv = require_csv_data_rows(
            route_csv,
            label='route waypoints',
            required_columns=('dist_kmh',),
        )
        self.route = pd.read_csv(route_csv)
        self.dt = float(self.get_parameter('dt').value)
        self.speed_kmh = float(self.get_parameter('init_speed_kmh').value)
        self.s_kmh = float(self.route['dist_kmh'].iloc[0])
        self.pub_gps = self.create_publisher(NavSatFix, '/sim/gps', 10)
        self.sub_speed = self.create_subscription(Float32, '/planner/speed_cmd', self.
on_speed, 10)
```

主要構造の読み取り

主要クラスは GPSSimNode。主要関数は on_speed, step, main。ROS パラメータ宣言は 3 件。ROS I/O は publisher 1 件、subscription 1 件。

主要クラス・関数

行	種別	名前	補足
11	class	GPSSimNode	bases: Node
12	function	__init__	args: self
32	function	on_speed	args: self, msg
34	function	step	args: self
47	function	main	

ファイルを上から読んだときの定義順

行	内容
11	クラス GPSSimNode を定義する。
47	関数 main を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
---	----------	-------------

1	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L48, L50。
2	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L11。
3	<code>from sensor_msgs. msg import NavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L28, L38。
4	<code>from std_msgs. msg import Float32</code>	Float32 や String など軽量メッセージで planner / vehicle 値を publish するため。このファイル内での主な使用位置は L29, L32。
5	<code>import pandas as pd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L24。
7	<code>from .route_ utils import interpolate_ route_with_alt</code>	route_utils.py から interpolate_route_with_alt を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/route_utils.py。このファイル内での主な使用位置は L37。
8	<code>from .path_ utils import resolve_path</code>	ROS share / 相対パス解決から resolve_path を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/path_utils.py。このファイル内での主な使用位置は L18。
9	<code>from .solar_ profile import require_csv_ data_rows</code>	profile YAML 読込と検証から require_csv_data_rows を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/solar_profile.py。このファイル内での主な使用位置は L19。

関数・クラスを上から順に解説

L11 クラス GPSSimNode

- 定義: `GPSSimNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `NavSatFix`, `__init__`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `float`, `get_clock`, `get_logger`, `get_parameter`, `info`, `interpolate_route_with_alt`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. 関数 `__init__` を定義する。
2. 関数 `on_speed` を定義する。
3. 関数 `step` を定義する。

L47 関数 main

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `GPSSimNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. `rclpy.init(...)` を実行する。

2. node に GPSSimNode() の結果を代入する。
3. rclpy.spin(...) を実行する。
4. node.destroy_node(...) を実行する。
5. rclpy.shutdown(...) を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
14	route_csv	inputs/route_waypoints.csv
15	dt	1.0
16	init_speed_kmh	40.0

ROS topic I/O

行	種別	topic	callback / 補足
28	publisher	/sim/gps	publisher
29	subscription	/planner/speed_cmd	self.on_speed

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/gps_sim_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。